



**NANYANG
TECHNOLOGICAL
UNIVERSITY**
SINGAPORE

NTU Academy for Professional
and Continuing Education

Context Engineering

Designing the full information window an LLM works inside

Module 5

Created by :

Swarup Biswas
May 2026



Prompt engineering asks: “How should I instruct the model?”

Context engineering asks: “What should the model know at this moment to perform the task well?”

You are the designer of the window

By the end of this hour you will have a mental model, a four-word diagnosis, four fixes, and one thing to build this week.

And you'll stop blaming the model.

An LLM is an operating system; the context window is its RAM

Computer concept	LLM equivalent	What it means in practice
CPU	The LLM itself	Processes only what is loaded into active memory
RAM	The context window	Fast, finite, lost when the session ends
Hard disk	Files, vector stores, databases	Persistent — but useless until paged into RAM
Operating system	System prompt + your context choices	Governs what loads, when, and in what order

Everything competes for the window — and the middle gets ignored

The window is a **finite budget**. System prompt, examples, tools, retrieved docs, memory, and history all compete for the same space — most people only design the last user turn.

Attention is **not** even: the **beginning and end** get the most; the **middle** gets the least — regardless of importance.

So put critical instructions at the **top AND bottom**, never buried in the middle.

Source: "Lost in the Middle", Liu et al. 2023 — reconfirmed on 1M-token models (Chroma, 2025)

Context fails in four ways — name which one

Mode	What happens	A picture
Poisoning	A wrong fact enters context and gets reused as truth	Wikipedia says Napoleon was born in 1969; uncaught, it's in your report
Distraction	The window is so full the signal drowns	20-page contract pasted; the payment terms are buried and vague
Confusion	Irrelevant content or too many tools sway the answer	15 tools attached; the agent keeps calling the wrong one
Clash	Parts of the context contradict each other	"Be concise but comprehensive" — you get something half-right

The fix isn't a bigger window — it's a cleaner one

Anthropic, Google, and the Manus team converged on this independently, 2025–2026.

A window **degrades as it fills up and goes stale** — more context is not more help. A 200K-token model can lose the plot by ~50K.

A bigger window just gives the cruft more room to accumulate. The fix is never "use a model with a bigger window" — it's **"design a cleaner one."**

Source: Chroma "Context Rot", 2025

Context rot: the detour doesn't clean up after itself

You're deep in **Task A**. A message pulls you to **Task B** for a few turns. You go back to A.

The window still carries **all of B** — and the model now quietly leans toward the newest topic

Pivot mid-conversation and performance drops ~**39%** — mostly **reliability**: once it takes a wrong turn, it doesn't recover

You can't cheaply un-see it. **A tangent means a new chat — not a return to the old one.**

Source: "LLMs Get Lost in Multi-Turn Conversation", Laban et al., arXiv 2505.06120

See it — less context, a sharper answer

Same model. Same question. The only change is the window.

Window A — paste the whole 20-page contract, ask "what are the payment terms?" → long, hedged, buried answer.

Window B — paste only the payment-terms clause, same question → short, exact, usable.

This is Distraction. The fix wasn't a bigger model — it was a cleaner window.

Every fix is one of four moves — WSCI

Select — choose what comes in: the right RAG chunks, examples, memory, tools. Bad selection can't be rescued by good compression.

Write — position and author: critical content at top or bottom, supersede don't stack, state which rule wins.

Compress — reduce what you keep: clear consumed tool results first (free, zero loss), then summarise or externalise to a note.

Isolate — separate what shouldn't mix: a fresh thread for a tangent, a separate window for a sub-task.

Each failure maps to the moves that fix it

Failure mode	Primary move	Secondary move
Poisoning	Select — verify & ground in sources	Isolate — validate at each handoff
Distraction	Select — include less	Compress — reduce what you include
Confusion	Select — fewer tools / less noise	Isolate — separate competing sources
Clash	Write — state which rule wins	Isolate — split contradictory content

Your turn — name the failure, name the fix

For each scenario, in pairs (60s): **which failure mode? which WSCI move fixes it?**

1. "Wikipedia said Napoleon was born in 1969. I didn't catch it and wrote it into my report."
2. "I pasted a 20-page contract and asked for the payment terms. The answer was buried and vague."
3. "My prompt says 'be concise but comprehensive.' I keep getting something half-right."
4. "I gave the agent 15 tools and it keeps calling the wrong one."
5. "A Slack ping made me switch topics mid-chat; now it's worse at my original task."

Habits that change how you work tomorrow

1. **Front-load the whole spec** — say the complete task once; don't drip requirements turn-by-turn (that's the 39% trap).
2. **One role, one task** — multiple roles average out; chain steps, don't combine them.
3. **Positive framing** — "write in flowing prose" beats "don't use bullets"; state the target.
4. **Fresh thread for tangents** — a detour leaves residue in the window (context rot); give a new topic a new window and bring back only the insight, never the whole conversation.
5. **Externalise durable facts** — put standing facts in a note or memory; start clean instead of nursing a bloated thread.

~65% of enterprise agent failures are context drift — not model capability.

Every criterion becomes a target the model games

Criterion in your prompt	How the model games it	Counter-driver to add
"Always give a complete answer"	Length stands in for completeness	"...only what the question requires, nothing it doesn't"
"Be helpful"	Volunteers unsolicited info and caveats	"Don't volunteer information that wasn't asked for"
"Be thorough"	Pads with edge cases and tangents	"Stop when the question is answered"

For every criterion: could a model satisfy it literally and still produce output you'd reject?

Six binary checks every shipped context must pass

Check	What to verify
1. Hierarchy	Does the prompt state which layer wins on conflict?
2. Positioning	Critical rules at top AND bottom — never buried?
3. Coverage	If every instruction is followed, does the output meet the need?
4. Gaming	Could each criterion be satisfied in a way you'd reject?
5. Few-shot edges	Examples for edge cases, not just happy paths?
6. Standalone test	Could a stranger predict behaviour from this context alone?

Your Monday task — one prompt, one audit, one portfolio piece

1. **Pick a real task** — a recurring email, report, or decision you actually face this week (not a textbook example).
2. **Draft a system prompt using WSCI** — select, write with hierarchy, compress, isolate.
3. **Run the six-check audit** — mark each check pass or fail.
4. **Add counter-drivers** — close every gaming path the audit flagged.
5. **Test with three diverse inputs** — capture the before-and-after. **That comparison is your portfolio piece.**

The names to drop in an interview — all on your study guide

WSCI · the **four failure modes** · **context rot** · **lost-in-the-middle**

mcp-scan / tool poisoning (MCP security) · **observability vs evals** · **Mem0 / Letta**
(agent memory)

Your study guide explains every one of these — read it this week; these are the keywords hiring managers screen for.

Design the context. Name the failure. Clean the window.

Reliable outputs follow. Take your worksheet — build one thing this week.

Version management

Month/Year	Version	Changes
May 2026	0.0	New module design (90-min)
Jul 2026	1.0	Upgraded slides - Redesigned to 60-min high-impact: 5 fundamentals, paired diagnostic drill, breadth moved to study guide;dedicated context-rot slide + needle-in-haystack framing